

软件分析论文中的“坏味道”

蒂姆·孟席斯

美国北卡罗来纳州立大学计算机科学系

马丁·谢泼德

布鲁内尔软件工程实验室 (BSEL)

伦敦布鲁内尔大学计算机科学系

英国伦敦 UB8 3PH 区

摘要

背景：为支持基于证据的软件工程，数据分析的使用迅速增加。然而，复杂的技术、多样的报告标准以及对潜在现象的理解不足，引发了人们对研究可靠性的担忧。

目标：我们的目标是软件分析研究（计算实验和相关性研究）的生产者和消费者提供指导。

方法：我们建议使用“坏味道”，即在敏捷软件开发社区中广为人知的深层次问题的表面迹象，并思考它们在软件分析研究中可能如何体现。

结果：我们列举了软件工程分析论文中的 12 种“坏味道”（并通过示例展示其影响）。

结论：我们认为“异味”这一隐喻是一种有用的工具。因此，我们鼓励就哪些因素有助于提高软件分析研究的有效性展开更多讨论（所以我们期望我们的列表会随着时间的推移而完善）。

1. 引言

自 Kitchenham 等人发表的开创性文章 [56] 以来，将软件工程确立为一门基于证据的学科的努力一直在不断推进。与此同时，公开可用的软件数据量和复杂的数据分析方法也大幅增长。这导致了基于实证、数据驱动的研究在数量和影响力上急剧增加，这些研究通常被称为软件分析。

通常，软件分析研究旨在将大量低价值数据提炼成少量高价值信息。这类研究更多是数据驱动而非理论驱动。例如，在研究了许多软件项目后，可以发现某些编码风格更容易出现错误。通过改变编码风格进行后续实验，可能会让我们相信其中存在因果关系。因此，我们可能会

建议避免某些编码风格（需视各种与上下文相关的注意事项而定）。然而，缺乏理论可能会带来挑战。特别是，由于缺乏强烈的先验信念，可能难以在潜在的数百万种可能性中进行选择或评估 [24, 28]。

到目前为止，一切顺利。不幸的是，软件工程（SE）领域内部 [93, 53] 以及更广泛的其他实验和数据驱动学科（如生物医学科学）[50, 31] 都对许多此类结果的可靠性表示担忧。这种情况已经发展到在实验心理学领域，研究人员现在提到了“可重复性危机”。这反过来又引发了研究有效性的问题。

可以说，软件工程领域的情况也并无不同。坎佩内斯等人在对软件工程实验（1993 - 2002 年）进行的一项重要系统综述中发现，所报告的效应量与心理学领域类似。2012 年，我们为《实证软件工程》杂志关于软件工程预测中可重复结果的一期特刊发表了一篇社论，其中主要关注的是“结论的不稳定性”。在那篇社论中，我们对众多这样的情况表示担忧：研究 1 得出结论 X，而研究 2 却得出结论 $\neg X$ 。无法解决这些看似矛盾的问题使我们陷入僵局。显然，我们需要更好的方法来开展和报告我们的工作，以及审视他人的工作。

在本文中，我们思考了可以从其他学科中学到什么，以及哪些具体经验教训应用于改进软件分析类型研究的报告。“坏味道”这一术语源自敏捷开发领域。根据福勒 [5] 的说法，坏味道（又称代码异味）是“一种表面迹象，通常对应着一个更深层次的问题”。本文所识别出的坏味道总结见表 1。

按照福勒的说法，我们认为本文所列的“问题迹象”并不一定意味着必须否定相关基础研究的结论。不过，我们认为这些迹象引发了三个潜在的关注方面。

1. 对于作者而言，它们降低了发表的可能性。
2. 对于读者而言，这些问题会让他们对结果的可靠性（即其一致性）和有效性（即分析的正确性，或其对我们认为它所代表的那些概念的把握程度）产生警惕。
3. 对于科学而言，它们阻碍了通过元分析汇总结果以及积累知识体系的机会。

因此，本文的目标是识别“异味”，以便（i）促进广泛的社区讨论，以及（ii）协助识别潜在问题及相关补救措施。

本文其余部分的结构安排如下。本节的剩余内容将首先阐述范围，然后处理关于这项工作的两个常见问题。第 2 节探讨了关于撰写“优秀”文章的历史研究和方法指导，以及其他数据驱动型学科如何解决实验可靠性问题。接下来，在第 3 节中，我们描述了识别“坏味道”的方法。随后，第 4 节列出了一些关键症状或大致的“味道”。

表 1: 软件分析坏味道总结

	"Bad smell"	Core problem	Impact	Remedy
1	Not interesting	The software analytics equivalent of how "many angels can dance on a pinhead". This may result from an absence of theory.	Research that has negligible software engineering impact	Dialogue with practitioners [6, 70, 51]
2	Not using related work	Unawareness of related work on (i) the research question and/or (ii) state of the art in relevant techniques.	(i) Unwitting duplication (ii) Not using state of the art methods (iii) Using unchallenging / outdated benchmarks	Read! Utilize systematic reviews whenever possible. Search for relevant theory or theory fragments. Utilize technical guidance on data analytics methods and statistical analysis. Speak to experts.
3	Using deprecated or suspect data e.g., D has been widely used in the past ...	Using data for convenience rather than for relevance	Data driven analysis is undetermined [2]	Justify choice of data sets wrt the research question.
4	Inadequate reporting	Partial reporting of results e.g., only giving means without measures of dispersion and/or incomplete description of algorithms	(i) Study is not reproducible [72] (ii) meta-analysis is difficult or impossible.	Provide scripts / code as well as data. Provide raw results [41, 79].
5	Under-powered studies	Small effect sizes and little or no underlying theory	Under-powered studies lead to over-estimation of effect sizes and replication problems	Analyse power in advance. Be wary of searching for small effect sizes, avoid Harking and p-hacking [15, 53, 79].
6	$p < 0.05$ and all that!	Over-reliance on, or abuse of, null hypothesis significance testing	A focus on statistical significance rather than effect sizes leading to vulnerability to questionable research practices and selective reporting [53]	Report effect sizes and confidence limits [33, 65].
7	Assumptions of normality and equal variances in statistical analysis	Impact of heteroscedasticity and outliers e.g., heavy tails ignored. Assumptions of analysis ignored.	Under-powered studies lead to over-estimation of effect sizes and replication problems.	Use robust statistics / involve statistical experts [58, 110].
8	No data visualisation	Simple summary statistics e.g., means and medians may mask underlying problems or unusual patterns.	Data are misinterpreted and anomalies missed.	Employ simple visualisations for the benefit of the analyst and the reader [80, 44].
9	Not exploring stability. Only reporting best or averaged results.	No sensitivity analysis. p-hacking	The impact of small measurement errors and small changes to the input data are unknown but potentially substantial. This is particularly acute for complex models and analyses.	Undertake sensitivity analysis or bootstraps/randomisation to understand variability. Minimally report all results and variances [73, 87].
10	Not tuning	Biased comparisons e.g., some models / learners tuned and others not. Non-reporting of the parameter settings, the tuning effort / expertise required.	Under-estimation of the performance of un-tuned predictors. Inability to reproduce results.	Read and use technical guidance on data analytics methods and statistical analysis [99, 37]. Unless relevant to the research question avoid off-the shelf defaults for sophisticated learners.
11	Not exploring simplicity	Excessively complex models, particularly with respect to sample size	Dangers of over-fitting	Independent validation sets or uncontaminated cross-validation [18].
12	Not justifying choice of learner	Permuting / recombining learners to make 'new' learners in a quest for spurious novelty	Under / non reporting of "uninteresting results" leading to over-estimates of effect sizes.	Principled generation of research questions and analysis techniques. Full reporting.

重要性顺序。最后，在第 5 节中，我们考虑对未来的影响，以及我们研究界如何共同掌控这些“代码异味”，重新确定优先级、重新排序、添加和去除“代码异味”。

1.1. 范围

软件分析研究相对较新且发展迅速，因此我们的很多建议都与讨论从软件项目数据中进行归纳的文章相关。我们通过实验、准实验和相关性研究来探讨实证数据分析中的问题 [91]，而不是例如定性研究相关的问题。关于定性研究以及涉及人类参与者的实验的技巧和陷阱，应参考其他文章，例如 [102, 25, 60, 84]。

最后，为了明确本文的范围，我们认为有必要对理论在软件分析类型研究中的作用补充一些说明。自然地，许多分析研究本质上是数据驱动或归纳性的。在这种情况下，理论或演绎推理的作用将是次要的。这当然引出了什么是理论的问题。不幸的是，除了理论可能包含概念、概念之间的关系（可能是因果关系）、范围以及可能的命题之外，对于什么构成理论，人们的共识有限 [97, 52]。大多数评论者认为理论的应用程度较低，或许可以有效地提高。作为朝着这个方向迈出的一步，斯托尔和菲茨杰拉德 [101] 探讨了理论片段的概念，这些片段可能以命题的形式出现。那些可检验的片段可能形成假设，即一种从理论到实证世界的机制，以及假定的命题，作为从实证观察回到理论的一种方式。

话虽如此，我们并未详细阐述软件工程理论。数据分析研究几乎总是理论性较弱，因为其方法是归纳性的。是否使用理论，在很大程度上取决于具体情境。目前对于理论（或模型）应该是什么样，尚无共识。最后，如前所述，已经有许多优秀的论述（见 [97, 52, 101]）。

1.2. 但这篇文章有必要吗？

简单来说：这篇文章有必要吗？这里的内容不都是“人人都知道”的材料吗？

对此，我们首先指出，在过去几十年里，作者们一直是许多会议和期刊评审小组的成员。基于这些经验，我们断言，显然并非每个人都了解（或至少应用）这些内容。每年，都有大量研究文章因违反本文的建议而被拒，更糟糕的是，有些文章还蒙混过关了。在我们看来，其中许多拒稿情况是可以避免的。我们认为，本文提出的许多问题在投稿前就可以解决，有些只需付出很少的努力。

这些困难因良好实践不断发展这一事实而更加复杂。例如，虽然许多从事实证软件工程研究的人员的统计教育以经典统计学为基础，但最近也有了一些新的发展，如稳健方法 [57]、随机化和自助法的使用 [16] 以及贝叶斯技术 [39]。同样，机器学习领域也有重大的最新发展。

学习内容，有时作者会忽略。例如，我们有时会审阅并拒掉一些论文，这些论文与我们其中一人 2007 年撰写的一篇文章 [76] 仅有细微差别。自那篇论文发表以来，情况发生了很大变化，所以仅仅在 2007 年那篇论文基础上进行微小增量，不太可能成为有价值的贡献或可接受的出版物。

我们还注意到，有时我们会发现一些担任科学出版物把关人角色的审稿人，似乎也没有意识到部分或大部分此类材料。因此，存在一些本不应发表（至少不应以当前形式发表）的研究却被发表了 [58, 53]。这损害了科学的进步，也损害了我们构建与软件工程相关的有用知识体系的能力。

一个特别边缘化的群体是试图在研究文献中发出自己声音的行业从业者。除非这个群体了解既定的出版规范，否则他们的文章有被拒稿的风险。我们注意到，这些出版规范很少有记录——这正是本文试图解决的一个问题。因此，基于上述所有原因，我们认为本文是必要且有用的。

1.3. 如果论文存在缺陷，是否应该被拒稿？

需要强调的是，本文：

- 主要是为作者提供指导；
- 并非供评审人员盲目地用作罗列拒稿潜在理由的清单。我们所列举的瑕疵只是提示性的，并非证明存在潜在问题的确凿证据。因此，我们强调，仅凭这些瑕疵的存在，并非评审人员拒稿的理由。具体语境极其重要。

本文的一位审稿人很好心地对这一点进行了详述。他们评论道（我们也认同），审稿人往往会变得过于严苛——甚至到了仅仅因为有一个点不符合某些准则，就可能拒绝那些包含非常有趣且重要观点的论文的程度。这在很少有修订周期的学术会议中最为突出。审稿人不应只是盲目地强制采用本文提出的所有要点，还需要对研究的背景保持敏感；某些“坏味道”可能并不适用（或者相关性较低）。此外，我们希望避免采取一种观点，即追求完美的要求会阻碍有用的、开创性的或相关的——但略有瑕疵的——研究成果发表。

1. 相关工作

软件工程（SE）研究人员时常提出这样一个问题：是什么造就了一篇好文章？例如肖（Shaw）在 2003 年发表的文章 [92]，泰森（Theisen）等人最近又重新探讨了这一问题 [107]。肖指出，SE 研究人员面临的一个困难是

¹ 例如，改进的评估技术和实验设计；软件工程和机器学习的新问题领域（例如 [88]）；新颖且有深刻见解的新评估指标（见 [49] 的第 5 节或 [81] 中讨论的多目标方法）；以及具有挑战性的新学习算法，其速度极慢，以至于重现先前的结果变得不切实际（见 [35] 中关于深度学习的讨论）。

而且作者们面临的问题是，对于如何开展和呈现研究，并没有完善或公认的指导方针。尽管在此期间，人们越来越强调证据，但我们认为情况并没有太大改变。她还指出，许多文章在验证方面存在不足，泰森等人报告称，即便有变化，也是评审人员的期望提高了。从潜在角度看，软件分析文章因其基于实证而在此方面具有优势，然而，正如我们的文章将要展示的，统计分析并非总能妥善进行。

也有对实证研究统计质量的各种审核 [58]，以及已发布的有关分析的指南和建议 [58, 111]。这些指南值得欢迎，因为总体而言，软件工程师接受的统计学正规训练有限。请注意，当研究人员发表方法指南时，这些文章往往引用率很高²，例如：

- 《软件工程实证研究初步指南》，发表于《IEEE 软件工程汇刊》[58]（自 2002 年以来被引用 1430 次）。
- 《软件缺陷预测的分类模型基准测试：一个提出的框架和新发现》，发表于 2008 年《IEEE 软件工程汇刊》[67]（自 2008 年以来被引用 765 次）。
- 《软件工程中案例研究的实施与报告指南》，发表于《实证软件工程》期刊 [86]（自 2009 年以来被引用 2500 次）。
- 《使用统计测试评估软件工程中随机算法的实用指南》，发表于 2011 年国际软件工程大会（ICSE’11）[4]（自 2011 年以来被引用 500 次）。

尤其值得关注的是我们重现研究结果的能力（或可靠性），许多人认为这是科学的基石。诺塞克等人 [83] 在实验心理学领域对这一问题进行了系统研究。他们从顶尖心理学杂志上 100 项实验的大规模重现项目中得出结论：“重现效应的强度仅为原始效应的一半”，并且虽然 97% 的原始研究取得了具有统计学意义的结果，但重现实验中只有 36% 做到了这一点。研究人员开始谈论“重现危机”。然而，需要注意的是，关于究竟什么才构成对原始研究的确证，目前讨论甚少 [100]，而且对于统计效力不足的研究（这似乎很常见），其结果很可能主要受抽样误差的影响。尽管如此，似乎确实有合理的理由令人担忧，这也引发了诸如穆纳福等人 [79] 撰写的近期“操作指南”类文章。尽管这种“重复危机”在被报道时引发了相当多的深刻反思，但取得的进展却比我们期望的要少。例如，在认知神经科学和实验心理学领域，研究效能仍然很低，并且在过去 50 年里基本没有变化 [104]。斯马尔迪诺和麦克尔雷思 [98] 指出，考虑到研究人员面临的激励机制——特别是发表论文是职业晋升的主要因素——低质量的研究并不令人惊讶。

² 这些引用次数数据提取自谷歌学术，2018 年 11 月。

仍在进行、提交和发表。正如他们所说，尽管这种情况并非“蓄意作弊或偷懒”，但这有助于解释为什么科学实践的改进仍然缓慢。

我们认为，虽然激励机制显然很重要，并会影响科学行为，但我们发现的许多问题只需付出相对较少的努力就能解决。我们也希望，“做好科学研究”的内在动力对我们绝大多数科研群体来说已经足够。

1. 我们的方法

我们的目标是帮助研究人员开展更出色的研究并撰写更优质的文章。为实现这一目标，我们提出一项包含两部分的提议：

1. 该提议的一部分是在主要的软件工程会议上举办研讨会，探讨并记录分析和撰写方面的最佳及最差实践。因此，本文旨在招募志同道合的同行，以实现这一目标，从而增加对这些问题进行思考和评论的研究人员数量。

2. 该计划的另一部分就是本文。我们希望这些观点本身能证明其价值，同时也希望：

(a) 本文能引发关于最佳和最差论文写作实践的更广泛讨论； (b) 后续文章能扩展并完善我们在下文提供的指导。

为使本文具有实际效果，我们采用务实的方法。我们列出了一份有关软件分析论文的问题清单，希望有所帮助，但并不详尽。细节程度以示例说明为主，而非深入探讨。本着简洁的原则，我们将重点限制在数据驱动的研究上，因此，例如涉及人类参与者的案例研究和实验不在讨论范围内。

有些“坏味道”更为明显，因此相对而言无需过多解释。相比之下，另一些则较为隐晦，所以我们会用更多篇幅来阐述。我们通过示例来帮助说明这些坏味道及其与实际问题的关系。我们举例子并非要证明其存在，而且无论如何，将个别作者挑出来作为普遍错误的例子，我们认为这是招人反感的。

这份“坏味道”清单是两位作者通过讨论和辩论制定的。然后，我们使用一种实证研究质量工具（见参考文献 [29]）对该清单（见表 2）进行了交叉核对，该工具在评估纳入软件工程系统评价的文章质量时会用到 [55，第 7 章]。因此，我们认为我们的清单涵盖了质量工具所确定的所有四个质量领域，对其覆盖范围更有信心。

1. 解析“不良迹象”

本节讨论表 1 中列出的不良现象。在开始之前，我们要指出，虽然以下一些“不良现象”特定于软件分析领域，但其他一些则普遍存在于许多科学研究中。我们将它们放在一起讨论，因为它们都是关乎良好科学研究和有效科学交流的重要问题。

表 2: 研究领域与不良现象的交叉引用

Research area [55]	Quality instrument question [29, Table 3]	Bad smell
<i>Reporting</i> - quality	Is the article based on research? Is there a clear statement of the research aims? Is there an adequate description of the context? Is there a clear statement of findings?	Yes, defined by our scope 1 4 4, 11
<i>Rigour</i> - details of the research design	Was the research design appropriate? Was the recruitment strategy appropriate? Treatments compared to a control group? Data collected addressed research issue? Was the data analysis sufficiently rigorous?	5 No human participants, but more generally: 1, 3 2, 10 3 6, 7, 8, 9, 10
<i>Credibility</i> - are the findings of the study are valid and meaningful?	Has the relationship between researcher and participants been adequately considered?	No human participants
<i>Relevance</i> - of the study to practice	Is the study of value for research or practice?	1, 11

4.1. 缺乏趣味性

所有科学研究都应努力做到有效且可靠。但同样重要的是，研究也应具有趣味性（面向比作者更广泛的受众）。然而，这并非是在呼吁开展离奇或不切实际的研究，要记住，我们的学科是软件工程。必须有某些人可能关注研究结果或潜在的研究结果。提高一篇文章被选中概率的一个经验法则是，既要保证有效性，又要吸引某些潜在受众。

那么我们所说的“有趣”是什么意思呢？文章需要有一个动机或信息，超越平凡。更具体地说，文章应该有一个观点。研究性文章应该为当前的争论做出贡献。这可以通过理论，或者针对其他研究的结果来实现。如果不存在这样的争论，研究性文章应该引发一场争论。例如，可以审视当前的文献，报告研究中需要调查的空白。但需要明确的是，这并不排除渐进式研究，甚至不排除验证其他研究人员的结果的可重复性，特别是当这些结果可能具有很强的现实意义时。

请注意，在这个数据海量可得且新机器学习算法层出不穷的时代，缺乏启发性的问题尤为突出；并非所有可能的研究方向实际上都有趣。当研究工作以数据驱动时，这种风险可能会特别高，因为这可能很快变成数据挖掘。关于这一点的更多内容，见我们最后提到的两个不良现象（“理论不足”和“理论不多”）。

考虑实际意义的一种有效的重要方法是使用效应量 [33]，尤其是当效应量能够以对目标受众有意义的方式表述时。

例如，通过 $x\%$ 或 y S 来降低单元测试成本，本质上比某些统计检验的相关 p 值低于任意阈值更具吸引力。例如，通过 $x\%$ 或 y S 来降低单元测试成本，本质上比某些统计检验的相关 p 值低于任意阈值更具吸引力。最小感兴趣效应量的概念也被提出作为一种机制，以保持对有趣的实证结果的关注 [64]，我们将回到这一点。

4.2. 未引用相关研究成果

这个问题本不应存在。然而，我们发现许多文章未能充分利用相关研究。通常，我们会看到以下几类具体的“不良现象”。

文献综述中没有或几乎没有发表时间距今少于十年的文章。当然，年代较早的论文可能非常重要，但完全没有近期的研究成果通常会令人惊讶。这有力地表明，该项研究并非处于前沿水平，或者未与前沿研究进行比较。

该综述忽略了相关的近期系统文献综述。这强烈表明，在阐述当前技术水平的方式上可能存在差距或误解。由于作者之前的工作可能会产生过度影响，这也可能导致偏见的产生。

这篇评论很肤浅，甚至有些敷衍了事。这种方式可能会被讽刺为：

“这里有一些相关工作 [1, ..., 30]。现在继续我描述自己工作的重要任务。”

这种写作风格——将评论他人的作品仅仅视为一种负担——降低了发现关联性的可能性，也降低了文章对知识体系有所贡献的可能性，相反，却增加了受证实性偏见影响的可能性 [82]。由于对其他作品几乎没有或完全没有解读，这种风格对读者也毫无帮助。哪些内容有用？哪些已被取代？

该综述全面、相关且公正，然而，它并未用于推动新的研究。换句话说，综述与新研究之间存在脱节。新结果与当前的技术水平并无关联。其后果是读者无法判断有哪些新的贡献。这也可能意味着错过了合适的基准。

该方法层面的依据源自极为简单化或过时的资料，例如使用传统统计学教材。使用这类资料的诸多风险之一，是支持已不再具有竞争力的过时或不恰当的数据分析技术或学习算法。

一个建设性的建议是，尤其是在没有相关系统文献综述的情况下，可以采用轻量级文献考察。这些考察不必像完整的系统文献综述那样复杂（完整的系统文献综述可能需要数月才能完成）。然而，随着已发表的系统文献综述数量不断增加，越来越有可能其他研究人员已经研究过与你正在探索的研究主题相关的至少部分文献。

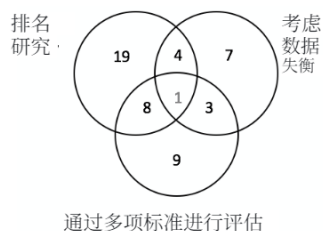


图 1: 表 3 的总结。

表 3: 21 项高被引的软件缺陷预测研究, 选取方式如下: (a) 在谷歌学术中, 搜索过去十年内包含 “软件” 缺陷预测” 面向对象 (OO)” 和 “CK 度量” 的内容; (b) 删除自发表以来每年引用次数少于 10 次的所有内容; (c) 在最终得到的 21 篇文章中, 查找 Agrawal 等人探讨的三个研究项目, 即它们是否对多个分类器进行排名; 是否依据多个标准对分类器进行排名; 该研究是否考虑类别不平衡问题。

Reference	Year	Cites	Ranked Classifiers?	using multiple criteria?	Considered Data Imbalance?
(a)	2007	855	✓	2	✗
(b)	2008	607	✓	1	✗
(c)	2008	298	✓	2	✗
(d)	2010	178	✓	3	✗
(e)	2008	159	✓	1	✗
(f)	2011	153	✓	2	✗
(g)	2013	150	✓	1	✗
(h)	2008	133	✓	1	✗
(i)	2013	115	✓	1	✓
(j)	2009	92	✓	1	✗
(k)	2012	79	✓	2	✗
(l)	2007	73	✗	2	✓
(m)	2007	66	✗	1	✓
(n)	2009	62	✓	3	✗
(o)	2010	60	✓	1	✓
(p)	2015	53	✓	1	✗
(q)	2008	41	✓	1	✗
(r)	2016	31	✓	1	✗
(s)	2015	27	✗	2	✓
(t)	2012	23	✗	1	✓
(u)	2016	15	✓	1	✗

关于轻量级文献调研的示例, 见阿格拉瓦尔 (Agrawal) 和门齐斯 (Menzies) [2] 的图 1 和表 3。在那里, 研究人员在过去十年中每年至少被引用 10 次的、引用次数最多的软件缺陷预测研究的二十多篇文章中 发现了一个研究空白。具体而言, 如图 1 所示, 大多数文章避开了 数据不平衡和多标准评分的问题。当然, 研究空白需要有吸引力 (见不良现象 #1)。这一发现促使阿格拉瓦尔和门齐斯开展了一系列独特的实验, 最终发表了一篇 2018 年国际软件工程大会 (ICSE' 18) 的文章。

4.3. 使用已弃用或可疑的数据

采用 高度数据驱动方法 的研究人员, 如软件分析领域的研究人员, 需要对数据质量保持警惕。德沃 (De Veaux) 和汉德 (Hand) 指出: 即使统计过程和动机是正确的, 糟糕的数据也可能产生完全无效的结果” [26]。一系列系统评价和映射研究 [68, 13, 85, 69] 均指出, 数据质量问题未得到足够重视, 而且在我们明确确定数据集的有效性方面存在问题, 尤其是当数据收集和分析之间的时间间隔 (可能还有地理间隔) 较大时。

就 “异味” 或警示信号而言, 我们建议研究人员关注以下几点:

- 非常陈旧的数据; 例如, 上世纪美国国家航空航天局 (NASA) 项目中的旧 PROMISE 数据 (JM1、CM1、KC1、KC2 等), 或 20 世纪 70 年代包含规模和工作量数据的原始 COCOMO 数据集。

- 有问题的数据；例如，已知存在质量问题的数据集（这同样包括上个世纪美国国家航空航天局（NASA）项目 [94] 中的旧 PROMISE 数据）。
- 来自简单问题的数据。例如，在软件测试研究的历史中，西门子测试套件是创建可复现研究问题的一个重要早期里程碑。受早期这项工作的启发，其他研究人员开始记录其他更复杂的问题³。因此，一篇将其所有结论都基于西门子套件中的三角不等式问题的新论文已过时。然而，小问题偶尔也有一定好处，例如，在尝试扩大规模之前，用于解释和调试算法。

话虽如此，我们的重点是软件工程领域的实证研究，在这个领域中，规模通常是个问题。根据我们的经验，读者越来越感兴趣的是来自更实际来源的结果，而不是来自合成问题或简单示例问题的结果。鉴于现在有许多在线存储库，研究人员可以在其中获取广泛的

来自多个项目（例如 GitHub、GitTorrent⁴；以及其他项目⁵）的一系列数据，这应该

这不会有问题。因此，现在使用从几十个到几百个项目中提取的数据进行发表是很常见的，例如 [61, 3]。

最后，我们承认，有时必须做出妥协，否则一些重要且有趣的软件工程问题可能永远无法得到解决。并非所有领域都有大规模、精心整理的高质量数据集。在这种情况下，研究可能会被视为更具探索性，或者在收集更好的数据方面起到某种激励作用。只要这些问题是透明的，我们认为就没有坏处。

4.4. 报告不充分

这里有两个维度：完整性和清晰度。问题可能通过以下“气味”表现出来。

1. 由于描述过于不完整或过于笼统，这项工作无法复现。此外，并非所有材料（数据和代码）都可获取 [41, 79]。
2. 这项工作无法精确复现，因为部分算法具有随机性且未提供随机种子。典型的例子有机器学习中交叉验证的折叠生成以及启发式搜索算法。
3. 不显著以及事后的“无趣”结果未被报告。这种看似无害的行为，即所谓的报告偏倚，可能导致对效应量的严重高估，并损害我们通过元分析或类似方法构建知识体系的能力 [41, 104]。
4. 实验细节中充斥着“神奇”数字，或者参数和实验设计的任意值。否则，对这些选择做出合理说明很重要。

³ 例如，见软件构件基础设施存储库 sir.unl.edu/portal。

⁴ github.com/cjb/GitTorrent

⁵ tiny.cc/seacraft

存在研究人员偏见和过度拟合的实际风险，即选择有利于他们钟爱的算法的值。标准的理由包括“ k 是通过标准 Gap 统计程序选择的”或进行一些敏感性分析，以探究选择不同值的影响 [87]。标准的理由包括“ k 是通过标准 Gap 统计程序选择的”。

- i. 如果不完整阅读文章，读者就不可能了解文章的主旨。按照本文所述的）结构化摘要能够极大地帮助人类读者 [14, 59, 11] 以及目前开始用于辅助元分析的文本挖掘算法，例如 [104]。
- j. 使用高度独特的章节标题或文章结构。通常，以下内容就足够了：引言、动机、相关工作、方法（分为数据、算法、实验方法和统计程序），接着是讨论的局限性、效度的局限性、结论、未来工作和参考文献。总结算法细节、超参数等的简单表格可能非常有用。
- k. 这篇文章使用了过多的首字母缩写词。在首次使用术语时进行定义。如果缩写词较多，可考虑使用术语表。

我们以一些简单且有望具有建设性的建议来结束这一小节。

力求实现可复现的结果：表 4 定义了研究工件和结果的连续统。报告可复现的结果（在该连续统的右侧）是理想情况，但这可能需要付出大量努力。关于如何构建研究工件以提高可重复性的精彩务实讨论，请参阅《科学计算的足够好实践》goo.gl/TD7Cax。

（旁白：对提高可重复性的要求必须基于务实的理由进行评估。如果先前的一项研究使用的工具不是开源的，或者是使用起来非常复杂的工具，那么要求后续工作完全重现先前的工作就不合适。）

考虑为每个表格或图注添加总结性文字，例如，图 X：统计结果表明只有少数学习者能在这项任务中取得成功”：通常，审稿人会先浏览一篇文章，只看图表。对于这样的读者来说，这些注释的局部性可能会非常有帮助。

考虑使用研究问题：通常，软件分析研究文章自然会细分为对一小部分研究问题的探索。确定这些研究问题需要一些创造力，但这是一种有效的组织手段。最好能有一定的递进关系，例如，以下是文献 [37] 中的研究问题（该文献讨论了调整数据挖掘算法超参数的优点）：

- 研究问题 1：调整参数是否能提高预测器的性能得分？
- 研究问题 2：调整（参数）是否会改变关于哪些学习者比其他学习者更优秀的结论？
- 研究问题 3：调整是否会改变关于哪些因素在软件工程中最为重要的结论？
- 研究问题 4：调整是否慢得离谱？

表 4：工件连续从左向右流动。一旦作者采用更多左侧的方法，就会有更多文章被归到右侧。此表展示了美国计算机协会（ACM）定义的徽章及定义（见 goo.gl/wVEZGx）。这些定义可供更广泛的群体用作：（1）理想目标，或（2）审核过往研究的方式，或（3）作为在会议或期刊上组织“工件专题”的指导。

No badge	Artifacts			Results	
	Functional	Reusable	Available	Replicated	Reproduced
					
	Artifacts documented, consistent, complete, executable, and include appropriate evidence of verification and validation.	Functional + very carefully documented and well-structured to the extent that reuse and re-purposing is facilitated. In particular, norms and standards of the research community for artifacts of this type are strictly adhered to.	Functional + placed on a publicly accessible archival repository. A DOI (Digital Object Identifier) or link to this repository along with a unique identifier for the object is provided. It is easy to obtain such DOIs: just host your artifact at repositories like SEACRAFT tiny.cc/seacraft .	Available + main results of the article have been obtained in a subsequent study by a person or team other than the authors, using, in part, artifacts provided by the author.	Available + the main results of the article have been independently obtained in a subsequent study by a person or team other than the authors, without the use of author-supplied artifacts.

• 研究问题 5：调优容易吗？

• 研究问题 6：数据挖掘工具是否应该直接使用默认调优设置“即开即用”？

一种有时与研究问题相关的有用排版手段，是使用结果框来突出问题的答案。以下是齐默尔曼等人 2004 年发表于文献 [114] 中的一个早期示例。尽管这类总结可能很有效，但需要精心构建，以避免过度简化而流于平庸。

人们要么可以得到精确的建议，要么可以得到大量的建议，但两者不可兼得。

4.5. 效力不足的研究

分析的功效是指一项检验能够拒绝错误的原假设的概率，即正确报告存在非零效应的概率。更普遍地说，它是指在存在效应时发现该效应的能力。功效是 β （即犯第二类错误的概率，也就是未能检测到真实效应而支持原假设，即出现假阴性）的互补概率，因此为 $power = 1 - \beta$ 。然而，这与犯第一类错误（即假阳性）的概率之间存在权衡。极端情况下，我们可以通过设定极高的接受阈值来防范假阳性，但后果是假阴性率会升高。反之亦然。

那么 β 应该设为多少呢？传统观点认为，功效应该设为 0.8 [21]，这意味着研究人员可能会考虑一类错误（错误地拒绝……）

原假设在现实中不存在影响时)的问题程度是第二类错误的四倍,因此需要相应更多的检验。因此,如果显著性水平设定为 $\alpha = 0.05$,那么 $\beta = 4\alpha = 0.2$,因此,按照要求,我们的统计功效为 $1 - \beta = 0.8$ 。再次强调,重要的一点是我们必须在第一类错误和第二类错误之间进行权衡,而且我们的偏好多少有些随意。

然而,我们期望一项研究的功效达到何种程度,与它实际的功效是两码事。有四个因素会影响功效,即: i) 样本量, ii) 所研究效应的未知真实大小, iii) 选定的 α 值, 以及 (iv) 选定的 β 值 [22, 33]。此外,实验设计可以提高功效,例如通过测量实验单位内部的差异(例如,采用重复测量设计),而非单位间的差异 [75]。

许多评论者建议,拟议的实验或调查的功效应事先确定 [30]。目的是检验一项实验是否有合理的机会检测到实际存在的效应。一个明显且直接的困难是估计要研究的效应量 [74]。由于报告和发表偏倚,我们在系统地高估效应量,有令人信服的证据表明存在这一问题,这使得情况更为复杂 [15, 53]。然而,功效问题不仅仅在于未能检测到实际存在的效应而浪费科学资源。还有另外两个严重的负面影响。第一是倾向于高估测量的效应量,第二是出现统计学上显著的假阳性结果的概率增加 [50, 23]。事实上,毫不夸张地说,低功效和功效不足的研究是对元分析和构建有意义的软件工程知识体系的最大威胁 [15, 104]。

具有讽刺意味的是,软件分析研究能够在庞大、复杂且杂乱的数据集中揭示模式和影响,而这一成功之处也正是其失败的原因。再加上缺乏指导性理论(这使得很难区分“赢家诅咒”与某些实质性影响⁶)、缺乏公认的分析方法以及研究人员自由度过度 [95, 71], 这些因素导致研究效力低下,并且极有可能高估效应量,使得其他研究人员难以复现研究结果 [50, 93, 53]。作为应对之策,我们强烈建议:

1. 预先确定感兴趣的最小效应量 (SESOL) [64]。这可能需要与问题所有者(通常是从业者)进行讨论。它还有助于减少开展无人关注的研究的可能性,从而解决我们的“不良现象 #1”。
2. 此外,提前进行功效计算,以确定该研究发现预期规模效应的可能性。在可能的情况下,如果有相关的既往研究,使用经过偏倚校正的合并估计值 [90],以帮助解决可能出现的高估问题。
3. 如果功效较低,可以修改研究设计 [75],使用更大的样本,或者

⁶ 黄等人 [48] 描述了基因组学中的一种类似情况,即有可能发现数百万种关联,但由于缺乏理论,研究人员几乎得不到指导。

研究一个不同的问题。

与这类问题相关的“气味”可能会出奇地难以察觉。不过，有一个线索可能是类似于这样的一种论点：

尽管样本量较小，但我们仍能够以极低的 p 值检测到一个极为显著的效应。因此可以得出结论，鉴于检测该效应存在固有困难，其实际效应肯定比最初显现的更为强烈。

不幸的是，事实并非如此。如果研究的统计效力不足，可能是由于样本量较小，那么极有可能出现的情况是，研究结果是假阳性，即犯了第一类错误 [71]。如果研究的统计效力较低，那就是一个问题，而且这个问题无法通过寻找较小的 p 值来解决。小样本带来的问题是，变异性会很大，只有那些偶然获得足够小的 p 值（因此满足通常为 $\alpha = 0.05$ 的接受标准）的研究，才会必然地对效应做出夸大的估计。这有时被称为“赢家诅咒”。达到统计显著性的观测效应实际反映真实效应的概率可以计算为 $([1 - \beta] \times R) / ([1 - \beta] \times R + \alpha)$ ，其中 $(1 - \beta)$ 是统计效力， β 是第二类错误， α 是第一类错误， R 是在所有被研究的效应中，所研究的效应真正非零的事前概率 [71, 104]。请注意，这个问题会引发下一个“不良信号”。

4.6. $p < 0.05$ 及诸如此类的情况！

尽管在很多文献中 [17, 96, 23] 都不提倡使用零假设显著性检验 (NHST)，但它仍然是统计分析的主流方法。因此，以下几个虚构的例子让人“感觉不对”：

我们的分析得出了具有统计学意义的相关性 ($r = 0.01$; $n = 18000$;
 $p = 0.049$; $\alpha = 0.05$)。

并且

表 X 表 X 总结了应用 ABC 推理检验得出的 500 个结果，并显示 40 在
 $\alpha = 0.05$ 这一常规阈值下具有统计学意义。

首先，较低的（且低于 $\alpha = 0.05$ 第一类错误接受阈值） p 值不应与实际的、现实世界中的效应混为一谈。 $r = 0.01$ 的相关性微不足道，实际上与无相关性并无区别。这仅仅是样本量极大 ($n = 18000$) 以及奈曼 - 皮尔逊假设检验 (NHST) 逻辑的产物，该逻辑要求将实证结果二分法划分为真实（存在非零效应）和虚假（效应量恰好为零）。因此，方法学家提倡关注最小效应量 (SESOI) 的概念。这应该在研究开展之前就明确。它还有一个优点

由于人们普遍认为使用 p 值存在的问题普遍存在 [96, 53]，因此我们认为挑出个人进行指责是令人反感的。

重新让研究人员关注他们试图解决的软件工程问题。比如说，对于软件工作量预测，在与问题负责人协商后，我们可能会得出结论，即特定环境下的目标（SESOI）是能够将平均误差降低 25%。一般来说（但不一定），效应量以标准化指标表示，以便在不同环境之间进行比较，因此上述特定环境下的目标（SESOI）可以通过观测值的变异性（标准差）进行归一化。可查阅 [33, 22] 获取通俗易懂的概述。如果文章能报告带有置信区间的效应量，这对从业者来说是有用的。毕竟，我们体验到的是效应，而非 p 值！

在“异味”的第二个例子中，问题在于进行过多的统计检验，并且只关注那些“显著”的结果。假设进行了 500 次检验，且我们接受错误拒绝原假设的概率为 1/20，那么仅靠随机数据，我们预计就会观察到 40 个“阳性”结果！已经提出了一系列的校正方法 [7]。使用本雅明尼 - 霍奇伯格方法 [8] 控制错误发现率可能是合适的，因为它校正了接受阈值，又不像传统的邦费罗尼校正那样严格 (α/t ，其中 t 是正在进行的检验次数)。

避免过度统计检验的一种方法是在进行统计分析之前对结果进行聚类。在这种方法中，不是问“哪些分布是不同的？”，而是在应用统计之前先应用某种分组操作。例如，考虑米塔斯（Mittas）和安杰利斯（Angelis）[78] 推荐的斯科特 - 诺特（Scott - Knott）程序。该方法根据中位数得分对具有 ls 次测量的 l 种处理的列表进行排序。然后，它将 l 划分为子列表 m 、 n ，以便在划分前后最大化观察到的性能差异的期望值。斯科特 - 诺特程序按如下方式确定其中一个划分为“最佳”。对于大小分别为 ls 、 ms 、 ns 的列表 l 、 m 和 n ，其中 $l = m \cup n$ ，“最佳”划分使 $E(\Delta)$ 最大化；即，划分前后的期望均值差异：

$$E(\Delta) = \frac{ms}{ls} \text{abs}(m.\mu - l.\mu)^2 + \frac{ns}{ls} \text{abs}(n.\mu - l.\mu)^2$$

然后，斯科特 - 诺特检验会检查“最佳”划分是否真的有用。为了确定这一点，该过程会应用一些统计假设检验 H ，以检查 m 和 n 是否有显著差异。如果有差异，斯科特 - 诺特检验会对“最佳”划分的每一半进行递归操作。

以一个具体例子来说，考虑来自 $l = 5$ 处理的结果：

$$r \times 1 = [0.34, 0.49, 0.51, 0.6]$$

$$r \times 2 = [0.6, 0.7, 0.8, 0.9]$$

$$r \times 3 = [0.15, 0.25, 0.4, 0.35]$$

$$r \times 4 = [0.6, 0.7, 0.8, 0.9]$$

$$r \times 5 = [0.1, 0.2, 0.3, 0.4]$$

经过排序和划分后，斯科特 - 诺特法确定：

- 排名第一的是 $rx5$ ，中位数为 0.25
- 排名第一的是 $rx3$ ，中位数为 0.3
- 排名第二的是 $rx1$ ，中位数为 0.5

treatment	Results from multiple runs								
no-SWReg	174	38	0	32	50	141	34	317	114
no-SLReg	127	73	45	35	47	159	37	272	103
no-CART-On	119	257	425	294	181	98	409	520	16
no-CART-Off	119	257	425	294	181	98	409	520	16
no-PLSR	63	213	338	358	163	44	14	520	44
no-PCR	55	240	392	418	176	45	12	648	65
no-1NN	119	66	81	82	70	110	118	122	92
log-SWReg	109	23	9	7	52	100	12	164	132
log-SLReg	108	61	49	53	55	156	28	183	101
log-CART-On	119	257	425	294	181	98	409	520	16
log-CART-Off	119	257	425	294	181	98	409	520	16
log-PLSR	163	7	60	70	2	152	31	310	175
log-PCR	212	180	145	155	70	68	1	553	293
log-1NN	46	231	246	154	46	23	200	197	158

注意：处理方法有一个预处理器（“无”表示无预处理器；“对数”表示使用对数变换）和一个学习器（例如，“SWReg”表示逐步回归；CART表示决策树学习器，在构建树后对其叶子进行后剪枝）。

图 2：数据展示的反面示例（在此例中，为误差百分比估计值，其中第一列中的处理方法应用于多个随机选择的数据分层）。请注意，a) 处理方法未按有趣程度从高到低排序；b) 未对分布情况进行总结；c) 未指明哪些值是最优的；以及 d) 统计上无显著差异的结果未归为一组。有关展示此数据的更好方法，请参见图 3。

–Ranked#3isrx2withmedian = 0.75

–Ranked#3isrx4withmedian = 0.75

请注意，Scott-Knott 检验发现 rx5 和 rx3 之间没有显著的有意义差异。因此，尽管它们的中位数不同，但它们的排名相同。关于此类分析的更详细示例，请参见图 2 和图 3。

斯科特 - 诺特程序可以改善过度统计检验的问题。要明白这一点，考虑对 10 种处理进行全对假设检验。这些处理可以通过 $(10^2 - 10)/2 = 45$ 种方式进行统计比较。为了大幅减少成对比较的数量，斯科特 - 诺特程序仅在找到能使性能差异最大化的划分之后才进行假设检验。因此，假设对整个数据进行二划分且所有划分都证明是不同的，那么斯科特 - 诺特程序将被调用 $\log_2(10) \approx 3$ 次。使用斯科特 - 诺特程序对这些划分进行 95% 置信度检验时，如果不对多次检验进行校正，其置信阈值将为： $0.95^3 = 86\%$ 。

使用自展法和非参数效应量检验（克里夫 δ ）来控制其递归双向聚类的实现可作为开源软件包获取，网址为 goo.gl/uo3FL。该软件包将图 2 这类数据转换为图 3 这样的报告。

4.7. 统计分析中的正态性和方差齐性假设

一个潜在的警示信号或“可疑之处”在于，在选择统计方法时，忽视数据的分布和方差，而只关注描述性的“重要”问题，例如使用均值等位置度量，以及应用 t 检验等推断统计方法。

现在人们普遍认识到，将传统的参数统计方法应用于违反正态性假设的数据，即这些数据与高斯分布有很大偏差。

RANK	TREATMENT	MEDIAN	(75-25)TH			
1	no-SWReg	50	107	- 0	-----	
1	log-SWReg	52	97	0	--	
1	log-SLReg	61	55	-0	----	
1	log-PLSR	70	132	- 0	-----	
1	no-SLReg	73	82	- 0	-----	
1	no-1NN	92	37		0	
2	log-PCR	155	142	---	0	-----
2	log-1NN	158	154	-	0	--
2	no-PLSR	163	294	--	0	-----
2	no-PCR	176	337	--	0	-----
3	log-CART-Off	257	290	-----	0	-----
3	log-CART-On	257	290	-----	0	-----
3	no-CART-On	257	290	-----	0	-----
3	no-CART-Off	257	290	-----	0	-----

图 3: 使用斯科特 - 诺特 (Scott-Knott) 方法, 有一种更好的方式来呈现图 2 中的错误结果 (错误值越低越好)。请注意, 在此处, 显然 “no-SWReg” 和 “CART” 处理分别表现最佳和最差。每种处理的结果按其中位数排序 (误差较小的前几行优于误差最大的最后几行)。结果以数字和可视化方式呈现 (右列标记第 10、30、50、70、90 百分位数范围; “0” 表示中值)。使用斯科特 - 诺特分析在第一列中对结果进行聚类。如果克里夫德尔塔 (Cliff's Delta) 报告的效应差异不止微小, 且自举检验报告存在显著差异, 则各行具有不同的等级。使用灰色背景来突出不同的等级。

分布可能会导致极具误导性的结果 [58]。然而, 有四点在软件工程界可能未得到足够重视。

1. 即使是微小的偏差也可能产生重大影响, 尤其是对于小样本而言 [110]。这些问题用诸如柯尔莫哥洛夫 - 斯米尔诺夫检验等传统测试方法并不总能轻易察觉, 因此应考虑通过 QQ 图进行可视化。
2. 样本方差之间的差异或异方差性, 可能比非正态性更成问题 [34]。因此, 普遍认为当样本量大致相等且大于 25 时, t 检验对偏离正态性具有稳健性, 这种观点并不保险 [34, 113]。
3. 尽管不同的变换, 例如对数变换和平方根变换, 可以减少不对称性和异常值的问题, 但它们很可能不足以处理其他违反假设的情况。
4. 最后, 在过去 30 年里, 稳健统计领域取得了重大进展, 通常比非参数统计更强大、更可取。这包括数据修剪和缩尾处理 [46, 57]。

由于软件工程项目 (SE) 数据集可能存在高度偏态、重尾分布或两者皆有, 我们建议进行稳健性检验以进行统计推断。请注意, 我们认为在任何可能的情况下, 稳健性检验都应优先于基于秩的非参数统计方法。例如, 后者中的威尔科克森符号秩检验 (Wilcoxon signed rank procedure) 可能效力不足, 并且在存在大量相同秩次的情况下会出现问题 [10]。Kitchenham 等人 [57] 在软件工程领域提供了有用的建议, 而 Erceg - Hurn 和 Mirosevich 给出了易于理解的一般性概述 [34]。如需广泛且详细的论述, 请参阅 [110]。

另一种方法是使用非参数自举法 (见埃弗龙 and 蒂布希拉尼 [32, 第 220 - 223 页])。请注意, 图 3 就是通过这样的自举检验生成的。使用

随着功能强大的计算机的出现，蒙特卡罗方法和随机化技术可以成为参数方法非常实用的替代方法 [73]。

4.8. 无数据可视化

虽然习惯上用一些汇总统计量（如均值、中位数和四分位数）来描述数据，但这并不一定能检查出异常模式和异常值。因此，我们强烈建议在进行统计检验之前先对数据进行可视化。例如：

- 图 3 的水平箱线图可以在一行中展示大量样本的结果。通过对这些数据进行直观检查，分析师可以判断其结论是否会因极端异常值或高方差等因素而受到质疑。

- 再举一个例子，考虑图 4，该图展示了一项 5 折交叉验证实验的结果，该实验探究了 SMOTE 的值：

- SMOTE 是一种数据平衡技术，它对训练数据进行调整，使任何类别都不会成为极少数类别 [2]。
- 5×5 交叉验证实验是一种评估技术，其中 $M = 5$ 次数，我们将数据顺序随机化。每次将项目数据划分为 N 个区间，在 $N - 1$ 个区间上进行训练，在留出的区间上进行测试（对所有区间重复此操作）。此过程产生 25 个结果，在图 4 中报告为中位数和四分位距（IQR）。

从这种直观检查来看，在进行任何统计分析之前，我们可以观察到，使用 SMOTE 在任何情况下都不会比不使用 SMOTE 更差。此外，如图 4 右侧的结果所示，应用 SMOTE 有时会带来非常大的提升。

4.9. 未探究稳定性

由于多种原因，软件工程数据集通常呈现出较大的差异 [77]。因此，重要的是要检查你想要报告的效果是否真实存在，还是仅仅是由噪声导致的。因此：

- 当有多个数据集可用时，尝试使用多个数据集。
- 报告集中趋势的度量（如中位数）和离散程度的度量（如标准差或四分位数间距（IQR，即第 75 百分位数 - 第 25 百分位数））。
- 不要仅仅对 N 个数据集的结果取平均值。相反，要总结结果在各个数据集上的分布情况，这可能包括创造性地使用图表，例如，见上文图 4。回顾该图，底部的虚线展示了图中每个数据集所显示的 25 个结果的变异性。这里的关键在于，这种变异性非常小；也就是说，中位数结果之间的差异往往远大于训练集随机选择所带来的变异性（当然，这种视觉印象需要合理验证）。

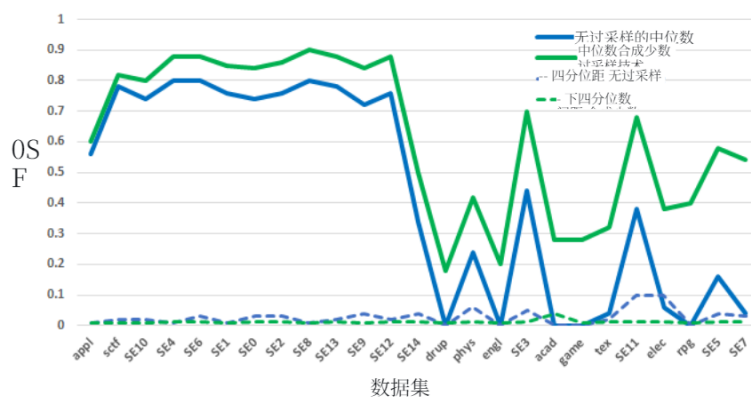


图 4: 对 25 个软件工程数据集进行文本挖掘时使用 SMOTE 算法的效果。在本图中, x 轴上的数据集按照使用和不使用 SMOTE 算法之间的差异大小进行排序。中位数和四分位距分别用实线和虚线表示 (回想一下, 中位数是第 50 个百分位数的结果, 四分位距是第 (75 - 25) 个百分位数的结果)。源自 [63]。

检查——这是统计检验的作用)。如果没有用于构建置信区间所需结果的离散统计量, 元分析将极其困难 [12, 90]。

- 如果仅探索一个数据集, 那么将分析重复 20 到 30 次 (然后通过自助法, 即有放回抽样, 来探索以这种方式生成的分布)。

• 时间验证: 假设数据被划分为一些时间序列 (例如, 同一项目的不同版本), 然后用过去的数据进行训练, 用未来的数据进行测试。我们在此建议保持未来测试集的大小不变; 例如,

在软件版本 $K-2, \dots, -1$ 上进行训练, 然后在版本 K 上进行测试。以此来验证。

增强了现实感, 并且很可能对该研究的目标用户更具吸引力。

- 跨项目验证: 在另一个项目的数据上进行训练, 然后在本项目上进行测试。请注意, 当不同项目使用具有不同属性名称或特征的数据时, 这种跨项目迁移学习可能会变得有些复杂。

• 项目内验证 (也称为 “交叉验证”): 将项目数据划分为 N 个区间, 在 $N - 1$ 个区间上进行训练, 在留出的区间上进行测试。对所有区间重复此过程。

请注意, 时间验证 (以及留一法交叉验证 LOOCV) 不会产生一系列结果, 因为目标始终是固定的。然而, 交叉验证和内部验证会产生结果的分布, 因此必须进行适当分析, 以便报告集中趋势的度量 (例如均值或中位数) 和离散程度的度量 (例如方差或四分位距)。作为一个普遍接受的规则, 为了获得, 比如说, 25 个跨项目学习的样本, 随机选择 90% 的数据, 重复 25 次。并且

获取，比如说，25 个项目内学习的样本，将数据顺序打乱 $M = 5$ 次，每次打乱后，运行 $N = 5$ 交叉验证。注意：对于比如说 60 个项目以下的小数据集，使用 $M, N = 8, 3$ （自 8 325 起）。

4.10. 不进行调优

许多机器学习人员，包括数据挖掘机，都选择了欠佳的默认设置 [108, 45, 37, 105, 1, 2]。最近大量研究结果表明，数据挖掘者的默认控制设置远非最优。例如，在进行缺陷预测时，多个团队报告称，通过调优能够找到新的设置，显著提升学习模型的性能 [37, 105, 106]。作为该调优的一个具体示例，考虑计算行 x 和行 y 之间距离的标准方法（其中每行包含 i 个变量）：

$$d(x, y) = \left(\sum_i \left((x_i - y_i)^2 \right) \right)^{1/2}$$

在使用 SMOTE 对数据类别进行重新平衡时，通过调优发现，将上述公式中的“2”值更改为其他值（通常是高达“3”的某个数字）很有用。

有一种特别恶劣的“风气”，就是将经过大量调优的新方法或热门方法与未经调优的其他基准进行比较。虽然从表面上看，新方法可能显得优越得多，但我们真正了解到的只是，把新技术用得好要比把另一种技术用得差表现更好。这种发现并没有特别大的用处。

至于如何进行调优，我们强烈不建议使用“网格搜索”，即一组嵌套的 for 循环，对所有可调整参数尝试广泛的设置。这是一种缓慢的方法；一项针对缺陷预测的网格搜索研究发现，它需要数天才能结束 [37]。不仅如此，有报告称网格搜索可能会错过重要的优化 [38]。每个网格在每个网格划分之间都有“间隙”，这意味着所谓严格的网格搜索仍可能错过重要的配置 [9]。伯格斯特拉 (Bergstra) 和本吉奥 (Bengio) [9] 评论说，对于大多数数据集，只有少数几个调优参数真正重要——这意味着与网格搜索相关的大部分运行时间实际上都被浪费了。更糟糕的是，伯格斯特拉和本吉奥观察到，不同数据集的重要调优参数是不同的，这种现象使得网格搜索在为新数据集配置软件分析算法时成为一个糟糕的选择。

我们建议不要使用“网格搜索”，非常简单的优化器（如差分进化 [103]）就足以找到更好的参数设置。这类优化器很容易从头开始编写代码。它们在诸如 jMETAL8 或 DEAP9 等开源工具包中也很容易获取。

4.11. 未探索简洁性

使用斯科特 - 诺特检验 (Scott-Knott tests) 的一个有趣且引人深思的方面是，通常，几十种处理方法可以仅被归为几个等级。例如，在文献 [40] 的斯科特 - 诺特检验结果中，32 个学习器仅被分为四组。

⁸jmetal.sourceforge.net

⁹github.com/DEAP/deap

虽然并非所有问题都能通过简单的方法解决，但将看似复杂的方法与一些更简单的方法进行比较是明智之举。通常，当我们对看似简单的方法进行编码和测试时，会发现它具有一些可取之处，从而让我们放弃更复杂的方法 [19, 36, 62]。此外，模型越简洁，就越不容易出现过拟合问题 [43]。

在当前的研究环境中，这种探索简单性的诉求尤为重要，因为当前的研究往往聚焦于深度学习，而深度学习实际上可能是一种非常缓慢的方法 [35]。对于具有高度复杂内部结构的问题，深度学习本质上更具优势 [66]。然而，许多软件工程数据集具有非常简单的规则结构，因此非常简单的技术能够执行得更快，并且表现与深度学习相当，甚至更好。这种更快的执行速度非常有用，因为它使先前结果的复现更加容易。

这并不是说深度学习与软件工程无关。相反，这意味着任何深度学习（或者就此而言，任何其他复杂技术）的结果也应该与一些更简单的基线（例如 [36]）进行比较，以证明额外的复杂性是合理的。例如，Whigham 等人 [109] 提出了一个基于自动数据转换和普通最小二乘法回归的简单基准作为比较对象，其理念是，如果复杂的方法无法超越简单的方法，就必须质疑其实际价值。

探索更简单选项的三种简单方法是侦察法、随机搜索法和特征选择法。霍尔特 (Holte) [47] 报告称，一种名为 1R 的非常简单的学习器可以在数据集中预先侦察，并在需要更复杂的学习时反馈。更普遍地说，通过首先运行一个非常简单的算法，很容易快速获得基线结果，从而了解问题的整体结构。

至于随机搜索，对于优化问题，我们经常发现随机搜索（例如，使用差分进化算法 [103]）在解决各种问题时表现都非常出色 [37, 1, 2]。请注意，霍尔特可能会说，对一个问题进行初始随机搜索就相当于对该问题进行侦察。

此外，对于分类问题，我们发现特征选择往往会发现大多数属性是冗余的，可以舍弃。例如，文献 [76] 中的特征选择实验表明，在保持预测能力的同时，41 个静态代码属性中多达 39 个可以被舍弃。文献 [20] 中也报告了类似的工作量估计结果，即大多数特征可以放心地忽略。这一点很重要，因为从较少特征中学习到的模型更易于阅读、理解、评判，也更容易向业务用户解释。此外，去除虚假特征后，研究人员得出虚假结论的可能性也会降低。再者，在降低数据维度后，任何后续处理都会更快、更容易。

请注意，许多文章都采用一种非常简单的线性时间特征选择方法，即：(i) 按相关性、方差或熵进行排序；(ii) 然后去除剩余特征中最差的一个；(iii) 用剩余特征构建一个模型；(iv) 如果新模型比之前训练的模型更差，则停止；(v) 否则，回到步骤 (ii)。霍尔 (Hall) 和霍姆斯 (Holmes) 报告称，这种贪婪搜索可能会过早停止，而通过并行增加有用特征集可以获得更好的结果（见他们基于相关性的特征子集选择算法，在 [42] 中有描述）。

在继续之前，我们注意到简单性研究的一个矛盾之处——它可能很难开展。在证明某些更简单的方法比最……

表 5: Ghotra 等人 [40] 发表于 ICSE' 15 的文章使用 Scott-Knott 程序对缺陷预测中常用的 32 种不同学习器进行了排名。这些排名分为四组。下表展示了这些组的一个示例。

RANK	LEARNER	NOTES
1 'best'	RF= random forest	Random forest of entropy-based decision trees.
	LR= Logistic regression	A generalized linear regression model.
2	KNN kth-nearest	Classify a new instance by finding k examples of similar instances. Ghotra et al. suggest $K = 8$.
	NB= Naive Bayes	Classify a new instance by (a) collecting mean and standard deviations of attributes in old instances of different classes; (b) return the class whose attributes are statistically most similar to the new instance.
3	DT= decision trees	Recursively divide data by selecting attribute splits that reduce the entropy of the class distribution.
4 'worst'	SVM= support vector machines	Map the raw data into a higher-dimensional space where it is easier to distinguish the examples.

复杂的最先进替代方案，可能需要将简单的方案与复杂的方案进行对比测试。这可能会导致一种奇特的情况，即可能需要花费数周的 CPU 时间来评估（比如说）某种非常简单的随机方法的价值，而该方法只需几分钟就能运行结束。

4.12. 未阐明对学习器的选择依据

这种风险，以及“异味”，源自机器学习技术的快速发展，并且构建不同学习器的复杂集合的能力意味着排列组合的数量正迅速变得极为庞大。将一个旧的学习器 L 轻易变换为 L' 是轻而易举的事，而且每个新的学习器都会产生一篇新文章。那么，这种“异味”就在于毫无动机地这样做，事先没有理由说明为什么我们可能会期望 L' 是有趣且值得探索的。

在评估某些新方法时，研究人员需要在一系列有趣的算法中对他们的方法进行比较。将所有新方法与所有先前的方法进行比较是一项艰巨的任务（因为所有先前方法的范围非常广泛）。相反，我们建议大多数研究人员对分析文献保持关注，寻找那些将表现似乎相似的学习器归为一类的重要论文，然后从每个聚类中至少选取一项来组成他们的比较集。

对于预测离散类别学习的学习器，[40] 的表 9 展示了一种这样的聚类。这是常用于缺陷预测的 32 个学习器的聚类。结果分为四组，最佳、次佳、次佳、最差（见表 5）。我们建议：

- 从每个组中随机选择一个作为你的比较算法范围；
- 或者从顶级组中选择所有的比较算法。

对于预测连续类别（即连续型变量）的学习器而言，目前尚不清楚什么是标准集，但随机森林回归树和逻辑回归得到了广泛应用。

对于基于搜索的软件工程问题，我们推荐一种名为 (you+two+next+dumb) 的实验技术，其定义为

- “你” 代表你的新方法；
- “Two” 指的是成熟且广泛使用的方法（通常是 NSGA-II 和 SPEA2 [89]）；
- 一种 “下一代” 方法，例如 MOEA/D [112]、NSGA-III [27]；
- 以及一种 “简单” 的基线方法（随机搜索或 SWAY [19]）。

i. 讨论

方法论的争论表明，研究人员群体在相互审视彼此的工作、检验彼此的假设并完善彼此的方法。在软件工程领域，我们认为，能从具体问题中抽身出来，去探讨 “什么样的文章才是一篇有效的文章？” 这一更普遍性问题的文章太少了。

为鼓励这一讨论，本文提出了 “坏味道” 的概念，将其作为一种利用表面症状来突出潜在问题的手段。为此，我们找出了十多种 “味道”。

本文认为，每年都有大量研究论文因违反本文的建议而被拒稿。其中许多拒稿是可以避免的——有些只需付出很少的努力——只要留意本文列出的不良现象即可。这一点对软件从业者尤为重要。除非遵循软件工程文章既定的规范，否则这一群体将很难在研究文献中发出自己的声音。我们羞愧地注意到，这些规范很少被记录下来——而本文正试图解决这一问题。

我们并不声称我们列出的不良气味是详尽无遗的——这是不可避免的。例如，我们在这里没有考虑的一个问题是评估使用不同评估标准的影响；或者研究软件分析方法的通用性。最近的研究 [62] 表明，看似最先进的技术，如为一个领域（缺陷预测）设计的迁移学习等任务，实际上在应用于其他领域（如代码异味检测等）时效果很差。尽管如此，本文的总体目标是鼓励就什么构成一篇 “有效” 的文章展开更多辩论，认识到有效性不是一个非黑即白的概念（因此加上引号）。我们希望在更广泛的研究社区中进一步展开辩论，使这个列表更加成熟。

我们认为，可以按如下方式展开这样一场辩论：

1. 多阅读，多思考。研究软件分析及其他领域所采用的方法。阅读文章时，不仅要考虑论点的内容，还要考虑其形式。
2. 推广并分享 “坏味道” 和反模式，以此作为分享不良实践的一种方式，但目的是推广良好实践。
3. 对研究进行预注册，并确定（实际）关注的最小效应量。

- l. 预先分析一项研究可能具有的功效。如果功效太低，修改研究设计（例如，收集更多数据或进行同类分析），或者更改研究内容。
- i. 力求研究结果具有可重复性。
- j. 制定、认可并采用适用于软件分析研究的行业标准。

进一步来说，正如我们上文所提到的，本文是制定软件分析社区标准这一战略计划的一部分。为此，我们希望本文能激发：

- 对最佳和最差论文写作实践展开更广泛的讨论；
- 一系列研讨会，研究人员齐聚一堂，就什么样的论文才算“好”论文展开辩论，并宣传我们学界的观点；
- 后续文章将对上述指南进行扩展和完善。

致谢

我们非常感谢编辑和审稿人提出的详细且有建设性的意见。

参考文献

参考文献

- [1] A. 阿格拉瓦尔、W. 傅、T. 门齐斯，〈主题建模存在什么问题？（以及如何使用基于搜索的软件工程方法解决）〉，CoRR 论文编号 abs/1608.08176，网址 <http://arxiv.org/abs/1608.08176>。
- [2] A. 阿格拉瓦尔、T. 门齐斯，〈更好的数据〉比“更好的数据挖掘算法”更优吗？论调整 SMOTE 算法对缺陷预测的益处，载于《第 40 届软件工程国际会议论文集》，美国计算机协会，2018 年，第 1050 - 1061 页。
- [3] A. 阿格拉瓦尔、A. 拉赫曼、R. 克里希纳、A. 索布兰、T. 门齐斯，〈我们不再需要英雄？“英雄”对软件开发的影响〉，计算机科学研究论文库（CoRR）论文编号 abs/1710.09055，网址 <http://arxiv.org/abs/1710.09055>。
- [4] A. 阿库里、L. 布赖恩德，〈软件工程中统计检验评估随机算法的实用指南〉，载于《第 33 届软件工程国际会议论文集》，ICSE '11，美国计算机协会，美国纽约州纽约市，国际标准书号 978 - 1 - 4503 - 0445 - 0，第 1 - 10 页，2011 年。
- [5] K. 贝克、M. 福勒、G. 贝克，〈代码中的坏味道〉，〈重构：改善既有代码的设计〉（1999 年），第 75 - 88 页。
- [6] A. 贝格尔、T. 齐默尔曼，〈分析一下！软件工程中数据科学家的 145 个问题〉，载于《第 36 届美国计算机协会软件工程国际会议论文集》，美国计算机协会，2014 年，第 12 - 23 页。

- [7] R. 本德、S. 兰格, 《多重检验的校正——何时及如何进行?》, 《临床流行病学杂志》, 2001 年第 54 卷第 4 期, 第 343 - 349 页。
- [8] Y. 本雅明尼, D. 耶库蒂埃利, 《相依情形下多重检验中错误发现率的控制》, 《统计年鉴》第 29 卷第 4 期 (2001 年), 第 1165 - 1188 页。
- [9] J. 伯格斯特拉、Y. 本吉奥, 《超参数优化的随机搜索》, 《机器学习研究杂志》2012 年 2 月第 13 卷, 第 281 - 305 页。
- [10] C. 布莱尔, J. 希金斯, 《不同总体分布下配对样本 t 检验与威尔科克森符号秩检验的功效比较》, 《心理学公报》97 卷 (第 1 期) (1985 年), 第 119 - 128 页。
- [11] A. 布斯、A. 奥罗克, 《从医学期刊数据库检索信息时结构化摘要的价值》, 《健康图书馆评论》14 卷第 3 期 (1997 年) 第 157 - 166 页。
- [12] M. 博伦斯坦、L. 赫奇斯、J. 希金斯、H. 罗斯坦, 《元分析导论》, 威利出版社, 英国西萨塞克斯郡奇切斯特, 2009 年。
- [13] M. 博苏、S. 麦克多内尔, 《实证软件工程中的数据质量: 有针对性的综述》, 载于《第 17 届软件工程评估与评价国际会议论文集》, 美国计算机协会, 2013 年, 第 171 - 176 页。
- [14] D. 巴登、A. 伯恩、B. 基奇纳姆, 《通过结构化摘要报告计算项目: 一项准实验》, 《实证软件工程》2011 年第 16 卷第 2 期, 第 244 - 277 页。
- [15] K. 巴顿、J. 约阿尼季斯、C. 莫克里兹、B. 诺塞克、J. 弗林特、E. 罗宾逊、M. 穆纳福, 《功效缺失: 小样本量为何会削弱神经科学的可靠性》, 《自然综述: 神经科学》2013 年第 14 卷第 5 期, 第 365 页。
- [16] J. 卡彭特, J. 比特尔, 《自助法置信区间: 何时用、用哪种、是什么? 医学统计学家实用指南》, 《医学统计学》19 卷第 9 期 (2000 年), 第 1141 - 1164 页。
- [17] R.P. 卡弗, 《反对统计显著性检验的案例, 再探讨》, 《实验教育杂志》61 卷 (第 4 期) (1993 年), 第 287 - 292 页。
- [18] G. 考利, N. 塔尔博特, 《关于模型选择中的过拟合及后续性能评估中的选择偏差》, 《机器学习研究杂志》11 卷 (第 7 期) (2010 年), 第 2079 - 2107 页。
- [19] J. 陈、V. 奈尔、R. 克里希纳、T. 门齐斯, 《“采样”作为基于搜索的软件工程的基线优化器》, 《IEEE 软件工程汇刊》。
- [20] Z. 陈、T. 门齐斯、D. 波特、D. 博姆, 《为软件成本建模寻找合适的数据》, 《IEEE 软件》22 卷 (6 期) (2005 年), 第 38 - 46 页。
- [21] J. 科恩, 《行为科学的统计功效分析》, 劳伦斯·厄尔鲍姆联合出版社, 新泽西州希尔斯代尔, 第 2 版, 1988 年。

- [22] J. 科恩, 《功效入门》, 《心理学公报》112 (1)(1992 年) 第 155 - 159 页。
- [23] D. 科尔库洪, 《错误发现率与 p 值的误读研究》, 《英国皇家学会开放科学》1 (140216)。
- [24] R. 考特尼, D. 古斯塔夫森, 软件度量中的随机抽样相关性, 《软件工程杂志》8 (1)(1993 年) 5 - 13 页。
- [25] D. S. 克鲁兹、T. 迪巴, 《软件工程中的研究综述: 一项三次研究》, 《信息与软件技术》53 卷 (5 期) (2011 年) 第 440 - 455 页。
- [26] R. 德·沃, D. 汉德, 《如何用错误数据说谎》, 《统计科学》20 卷第 3 期 (2005 年) 第 231 - 238 页。
- [27] K. 德布、H. 贾因, 一种基于参考点非支配排序法的进化多目标优化算法, 第一部分: 求解盒约束问题, 《电气与电子工程师协会进化计算汇刊》18 卷第 4 期 (2014 年), 577 - 601 页。
- [28] P. 德万布、T. 齐默尔曼、C. 伯德, 《实证软件工程中的信念与证据》, 载于《IEEE/ACM 第 38 届软件工程国际会议 (ICSE) 论文集》, 电气与电子工程师协会, 2016 年, 第 108 - 119 页。
- [29] T. 迪巴、T. 丁瑟尔, 《软件工程系统评价中的证据强度》, 载于《第二届 ACM - IEEE 实证软件工程与测量国际研讨会论文集》, ACM 出版社, 2008 年, 第 178 - 187 页。
- [30] T. 迪巴、V. 坎佩内斯、D. 舍贝里, 《软件工程实验中统计功效的系统综述》, 《信息与软件技术》48 卷 (第 8 期) (2006 年), 第 745 - 755 页。
- [31] B. 厄普、D. 特拉菲莫夫, 《复制、证伪与社会心理学的信任危机》, 《心理学前沿》2015 年第 6 卷, 第 621 - 632 页。
- [32] B. 埃弗龙, R. J. 蒂布希拉尼, 《自助法导论》, 《统计与应用概率专著》, 查普曼与霍尔出版社, 伦敦, 1993 年。
- [33] P. 埃利斯, 《效应量必备指南: 统计功效、元分析与研究结果解读》, 剑桥大学出版社, 2010 年。
- [34] D. 埃尔采格 - 赫恩、V. 米罗塞维奇, 《现代稳健统计方法: 一种最大限度提高研究准确性和效能的简便途径》, 《美国心理学家》2008 年第 63 卷第 7 期, 第 591 - 601 页。
- [35] W. 傅、T. 孟席斯, 《易先于难: 深度学习的案例研究》, 载于《2017 年第 11 届软件工程基础联合会议论文集》, 美国计算机协会, 2017 年, 第 49 - 60 页。
- [36] 傅 W.、门齐斯 T., 《再探用于缺陷预测的无监督学习》, 载于《2017 年第 11 届软件工程基础联合会议论文集》, 美国计算机协会, 2017 年, 第 72 - 83 页。

- [37] 付伟、T. 孟席斯、沈晓等, 《软件分析的调优》, 《信息与软件技术》2016 年第 76 卷 (C) 第 135 - 146 页, 国际标准连续出版物编号 0950 - 5849。
- [38] W. 傅、V. 奈尔、T. 孟席斯, 《为何差分进化在调整缺陷预测器方面优于网格搜索? 》, CoRR 论文编号 abs/1609.02613, 网址 <http://arxiv.org/abs/1609.02613>。
- [39] A. 格尔曼、J. 卡林、H. 斯特恩、D. 邓森、A. 韦塔里、D. 鲁宾, 《贝叶斯数据分析》, CRC 出版社, 第 3 版, 2013 年。
- [40] B. 戈特拉、S. 麦金托什、A. E. 哈桑, 《重新审视分类技术对缺陷预测模型性能的影响》, 载于《第 37 届软件工程国际会议论文集 - 第 1 卷》, IEEE 出版社, 2015 年, 第 789 - 800 页。
- [41] S. 古德曼、D. 法内利、J. 约阿尼季斯, 研究可重复性意味着什么?, 《科学转化医学》8 卷 (341 期) (2016 年) 341ps12 - 341ps12。
- [42] M. A. 霍尔、G. 霍姆斯, 离散类别数据挖掘中属性选择技术的基准测试, 《IEEE 知识与数据工程汇刊》15 卷第 6 期 (2003 年), 第 1437 - 1447 页。
- [43] D. 霍金斯, 《拟合问题》, 《化学信息与计算机科学杂志》44 (1)(2004) 1 - 12。
- [44] K. 希利, 《数据可视化: 实践入门》, 普林斯顿大学出版社, 2018 年。
- [45] H. 赫罗多托、H. 林、G. 罗、N. 鲍里索夫、L. 董、F. B. 切廷、S. 巴布, 《海星: 一种面向大数据分析的自调优系统》, 发表于《CIDR》, 第 11 卷, 第 261 - 272 页, 2011 年。
- [46] D. 霍格林、F. 莫斯特勒、J. 图基, 《理解稳健性与探索性数据分析》, 约翰·威利父子出版公司, 1983 年。
- [47] R. 霍尔特, 《非常简单的分类规则在大多数常用数据集上表现良好》, 《机器学习》11 (1)(1993 年) 63 - 90 页, 国际标准连续出版物编号 1573 - 0565。
- [48] 黄 Q.、里奇 S.、布罗津斯卡 M.、井上 M., 《QTL 研究中的功效、错误发现率与赢家诅咒》, bioRxiv (2017 年), 第 209171 页。
- [49] 黄 Q., 夏 X., 罗 D., 《监督模型与无监督模型: 从整体视角看工作量感知的即时缺陷预测》, 载于: 2017 年 IEEE 国际软件维护与演化会议 (ICSME), IEEE, 第 159 - 170 页, 2017 年。
- [50] J. 约阿尼季斯, 《为什么大多数已发表的研究结果都是错误的》, 《公共科学图书馆·医学》2 (8) (2005 年) e124 - 126 页。
- [51] M. 伊瓦尔松、T. 戈尔舍克, 一种评估技术评估的严谨性和行业相关性的方法, 《实证软件工程》16 (3)(2011) 365 - 395。

- [52] P. 约翰逊、M. 埃克施泰特、I. 雅各布森, 《软件工程的理论在哪里?》, 《IEEE 软件》2012 年第 29 卷第 5 期, 第 96 - 96 页。
- [53] M. 约根森、T. 迪巴、K. 利斯特尔、D. 舍贝格, 《软件工程实验中的错误结果: 如何改进研究实践》, 《系统与软件杂志》2016 年第 116 卷, 第 133 - 145 页。
- [54] V. 坎佩内斯、T. 迪博、J. 汉内、D. 舍贝格, 《软件工程实验中效应量的系统评价》, 《信息与软件技术》49 卷 (11 - 12 期) (2007 年), 第 1073 - 1086 页。
- [55] B. 基奇纳姆、D. 巴登、P. 布雷顿, 《验证软件工程与系统评价》, 美国佛罗里达州博卡拉顿 CRC 出版社, 2015 年。
- [56] B. 基奇纳姆、T. 迪巴、M. 约根森, 《基于证据的软件工程》, 载于: 第 26 届 IEEE 国际软件工程会议 (ICSE 2004), IEEE 计算机学会, 2004 年, 第 273 - 281 页。
- [57] B. 基奇纳姆、L. 马德伊斯基、D. 巴登、J. 基恩、P. 布雷顿、S. 查特斯、S. 吉布斯、A. 波通, 《验证软件工程的稳健统计方法》, 《验证软件工程》22 (2)(2017) 579 - 630。
- [58] B. 基奇纳姆、S. 普莱格、L. 皮卡德、P. 琼斯、D. 霍格林、K. 埃尔·埃马姆、J. 罗森伯格, 《软件工程实证研究初步指南》, 《IEEE 软件工程汇刊》28 卷 (第 8 期) (2002 年), 第 721 - 734 页。
- [59] B. A. 基奇纳姆、O. P. 布雷顿、S. 欧文、J. 布彻、C. 杰弗里斯, 《结构化软件工程摘要的长度与可读性》, 《ET 软件》2 卷第 1 期 (2008 年), 第 37 - 45 页。
- [60] A. J. 科、T. D. 拉托扎、M. M. 伯内特, 《面向有人类参与者的软件工程工具受控实验实用指南》, 《验证软件工程》20 (1)(2015 年) 第 110 - 141 页。
- [61] R. 克里希纳、A. 阿格拉瓦尔、A. 拉赫曼、A. 索布兰、T. 门齐斯, 《问题、缺陷与增强功能之间有何联系? (800 多个软件项目的经验教训)》, CoRR 论文编号 abs/1710.08736, 网址 <http://arxiv.org/abs/1710.08736>。
- [62] R. 克里希纳、T. 门齐斯, 《更简单的迁移学习 (使用 “领头羊”)》, arXiv 预印本 abs/1703.06218, 网址 <http://arxiv.org/abs/1703.06218>。
- [63] R. 克里希纳、Z. 于、A. 阿格拉瓦尔、M. 多明格斯、D. 沃尔夫, “BigSE” 项目: 工业文本挖掘验证的经验教训, 2016 年电气与电子工程师协会 / 计算机协会第二届大数据软件工程国际研讨会 (BIGDSE) (2016 年) 第 65 - 71 页。
- [64] D. 拉肯斯, 《等效性检验: t 检验、相关性和元分析实用入门》, 《社会心理与人格科学》2017 年第 8 卷第 4 期, 第 355 - 362 页。

- [65] D. 拉肯斯、E. 埃弗斯, 《从混沌之海驶向稳定走廊: 提高研究信息价值的实用建议》, 《心理科学视角》9 (3)(2014 年) 第 278 - 292 页。
- [66] Y. 勒昆、Y. 本吉奥、G. 欣顿, 《深度学习》, 《自然》521 卷 (7553 期) 2015 年) 第 436 页。
- [67] S. 莱斯曼、B. 贝森斯、C. 穆斯、S. 皮茨希, 《软件缺陷预测分类模型的基准测试: 一个提议的框架和新发现》, 《IEEE 软件工程汇刊》34 卷第 4 期 (2008 年), 第 485 - 496 页。
- [68] G. 利布申、M. 谢泼德, 《软件工程中的数据集与数据质量》, 载于《PROMISE 2008》, 美国计算机协会出版社, 2008 年。
- [69] G. 利布申、M. 谢泼德, 《软件工程中的数据集与数据质量: 八年回顾》, 载于《第 12 届软件工程预测模型与数据分析国际会议论文集》, 美国计算机协会, 2016 年。
- [70] D. 罗、N. 纳加潘、T. 齐默尔曼, 《从业者如何看待软件工程研究的相关性》, 载于《2015 年第 10 届软件工程基础联合会议论文集》, 美国计算机协会, 2015 年, 第 415 - 425 页。
- [71] E. 洛肯、A. 格尔曼, 《测量误差与再现性危机》, 《科学》355 卷 (6325 期) (2017 年), 第 584 - 585 页。
- [72] L. 马德伊斯基、B. 基奇纳姆, 《更广泛地采用可重复性研究对实证软件工程研究有益吗?》, 《智能与模糊系统杂志》32 卷 (2017 年) 第 1509 - 1521 页。
- [73] B. 曼利, 《生物学中的随机化、自助法与蒙特卡罗方法》, 查普曼与霍尔出版社, 伦敦, 1997 年。
- [74] S. 麦克斯韦尔、K. 凯利、J. 劳施, 《参数估计中统计功效与准确性的样本量规划》, 《心理学年鉴》59 卷 (2008 年) 第 537 - 563 页。
- [75] G. 麦克莱兰, 《在不增加样本量的情况下提高统计功效》, 《美国心理学家》2000 年第 55 卷第 8 期, 第 963 - 964 页。
- [76] T. 孟席斯、J. 格林沃尔德、A. 弗兰克, 《通过挖掘静态代码属性学习缺陷预测器》, 《IEEE 软件工程汇刊》33 卷第 1 期 (2007 年), 第 2 - 13 页。
- [77] T. 孟席斯、M. 谢泼德, 社论: 软件工程预测中可重复结果特刊, 2012 年。
- [78] N. 米塔斯、L. 安杰利斯, 《通过多重比较算法对软件成本估算模型进行排序和聚类》, 《IEEE 软件工程汇刊》39 卷 (4 期) 2013 年) 537 - 551 页。

- [79] M. 穆纳福、B. 诺塞克、D. 毕晓普、K. 巴顿、C. 钱伯斯、N. 迪塞尔、U. 西蒙松、E. 瓦根 makers、J. 韦尔、J. 约阿尼迪斯, 《重复性科学宣言》, 《自然·人类行为》1 卷 (2017 年), 0021 页。
- [80] G. 迈亚特, W. 约翰逊, 《数据解读 II: 数据可视化、高级数据挖掘方法及应用实用指南》, 约翰·威利父子出版公司, 2009 年。
- [81] V. 奈尔、A. 阿格拉瓦尔、J. 陈、W. 傅、G. 马修、T. 门齐斯、L. 明库、M. 瓦格纳、Z. 于, 《数据驱动的基于搜索的软件工程》, 载于: 2018 年 MSR 会议, 2018 年。
- [82] R. 尼克森, 《确认偏差: 一种以多种形式普遍存在的现象》, 《普通心理学评论》第 2 卷第 2 期 (1998 年), 第 175 - 220 页。
- [83] 开放科学合作组织, 《心理科学再现性的评估》, 《科学》349 卷 (6251 期) (2015 年), aac4716 - 3 页。
- [84] K. 彼得森、S. 瓦卡拉兰卡、L. 库兹尼亚尔兹, 《软件工程中进行系统映射研究的指南: 更新版》, 《信息与软件技术》64 卷 (2015 年) 第 1 - 18 页。
- [85] M. 罗斯利、E. 坦佩罗、A. 勒克斯顿 - 赖利, 我们能相信我们的结果吗? 一项关于数据质量的映射研究, 载于: 第 20 届 IEEE 亚太软件工程会议 (APSEC' 13), 2013 年, 第 116 - 123 页。
- [86] P. 鲁内松、M. 赫斯特, 《软件工程中案例研究的实施与报告指南》, 《实证软件工程》第 2 卷第 14 期 (2009 年), 第 131 - 164 页。
- [87] A. 萨尔泰利、S. 塔兰托拉、F. 坎波隆戈, 《敏感性分析作为建模的一个要素》, 《统计科学》15 卷 (4 期) (2000 年), 第 377 - 395 页。
- [88] A. 萨卡尔、J. 郭、N. 西格蒙德、S. 阿佩尔、K. 恰尔内茨基, 《配置系统性能预测的成本效益采样 (t)》, 载于《自动化软件工程 (ASE)》, 2015 年第 30 届 IEEE/ACM 国际会议论文集, IEEE, 2015 年, 第 342 - 352 页。
- [89] A. S. 赛义德、H. 阿马尔, 基于帕累托最优搜索的软件工程 (POSBSE): 文献综述, 载于: 第二届实现软件工程中人工智能协同效应国际研讨会 (RAISE), 电气与电子工程师协会, 2013 年, 第 21 - 27 页。
- [90] F. 施密特、J. 亨特, 《分析方法: 纠正研究结果中的误差与偏差》, 塞奇出版社, 第 3 版, 2014 年。
- [91] W. 沙迪什、T. 库克、D. 坎贝尔, 《广义因果推断的实验设计与准实验设计》, 霍顿·米夫林出版公司, 波士顿, 2002 年。
- [92] M. 肖, 《撰写优秀的软件工程研究论文》, 载于《第 25 届 IEEE 国际软件工程会议论文集》, 电气与电子工程师协会计算机学会, 2003 年, 第 726 - 736 页。

- [93] M. 谢泼德、D. 鲍斯、T. 霍尔, 《研究人员偏差: 机器学习在软件缺陷预测中的应用》, 《IEEE 软件工程汇刊》40 卷(6 期) 2014 年, 第 603 - 616 页。
- [94] M. 谢泼德、Q. 宋、Z. 孙、C. 迈尔, 《数据质量: 关于美国国家航空航天局软件缺陷数据集的一些评论》, 《IEEE 软件工程汇刊》39 卷(第 9 期) 2013 年 1208 - 1215 页。
- [95] R. 西尔伯察恩、E. 厄尔曼、D. 马丁、P. 安塞尔米、F. 奥斯特、E. 奥特里、Š. 巴尼克、F. 白、C. 班纳德、E. 博尼耶等人, 众多分析者, 一个数据集: 揭示分析选择的差异如何影响结果, 网址<https://osf.io/j5v8f>。
- [96] J. 西蒙斯、L. 纳尔逊、U. 西蒙松, 《心理学中的假阳性: 数据收集与分析中未披露的灵活性使任何结果都看似显著》, 《心理科学》22 卷(第 11 期) 2011 年, 第 1359 - 1366 页。
- [97] D. 舍贝格、T. 迪博、B. 安达、J. 汉内, 《软件工程中的理论构建》, 载于《高级实证软件工程指南》, 施普林格出版社, 2008 年, 第 312 - 336 页。
- [98] P. 斯马尔迪诺、R. 麦克里思, 《糟糕科学的自然选择》, 《英国皇家学会开放科学》3 卷(第 9 期) 2016 年, 第 160384 页。
- [99] J. 斯诺克、H. 拉罗谢尔、R. 亚当斯, 《机器学习算法的实用贝叶斯优化》, 收录于《神经信息处理系统进展》NIPS 2012 年, 第 2951 - 2959 页, 2012 年)。
- [100] J. 斯彭斯、D. 斯坦利, 《预测区间: 当你有所期待时该期待什么…… 一项重复研究》, 《公共科学图书馆·综合》11 卷(第 9 期) 2016 年, e0162874。
- [101] K.-J. 斯托尔、B. 菲茨杰拉德, 《软件工程理论的探索》, 载于: 第二届软件工程通用理论(GTSE) SEMAT 研讨会论文集, 电气与电子工程师协会(IEEE), 2013 年, 第 5 - 14 页。
- [102] K.-J. 斯托尔、P. 拉尔夫、B. 菲茨杰拉德, 《软件工程研究中的扎根理论: 批判性回顾与指南》, 载于《软件工程(ICSE), 2016 年 IEEE/ACM 第 38 届国际会议》, IEEE, 2016 年, 第 120 - 131 页。
- [103] R. 斯托恩、K. 普赖斯, 《差分进化——一种求解连续空间全局优化问题的简单高效启发式算法》, 《全局优化杂志》11 (4)(1997) 341 - 359, 国际标准连续出版物编号 1573 - 2916。
- [104] D. 祖克斯、J. 约阿尼季斯, 近期认知神经科学与心理学文献中已发表效应量和效力的实证评估, 《公共科学图书馆·生物学》15 卷(3 期) 2017 年, e2000797。
- [105] C. 坦蒂坦塔沃恩、S. 麦金托什、A. 哈桑、K. 松本, 缺陷预测模型分类技术的自动参数优化, 见: 第 38 届 IEEE/ACM 国际软件工程会议(ICSE 2016), IEEE, 2016 年, 第 321 - 332 页。

- [106] C. 坦蒂坦塔沃恩、S. 麦金托什、A. E. 哈桑、K. 松本, 《自动参数优化对缺陷预测模型的影响》, 《IEEE 软件工程汇刊》。
- [107] C. 泰森、M. 杜奈斯基、L. 威廉姆斯、W. 维瑟, 《撰写优秀的软件工程研究论文: 再探讨》, 载于《IEEE/ACM 第 39 届软件工程国际会议会刊 (ICSE-C)》, 电气与电子工程师协会, 2017 年, 第 402 - 402 页。
- [108] D. 范·阿肯、A. 帕夫洛、G. 戈登、B. 张, 《通过大规模机器学习实现自动数据库管理系统调优》, 载于《2017 年美国计算机协会数据管理国际会议论文集》, 美国计算机协会, 2017 年, 第 1009 - 1024 页。
- [109] P. 惠格姆、C. 欧文、S. 麦克唐奈, 《一种软件工作量估算的基线模型》, 《美国计算机协会软件工程与方法学汇刊》第 24 卷第 3 期。
- [110] R. 威尔科克斯, 《稳健估计与假设检验导论》, 学术出版社, 第 3 版, 2012 年。
- [111] C. 沃林、P. 鲁内松、M. 赫斯特、M. 奥尔松、B. 雷格内尔、A. 韦斯林, 《软件工程中的实验》, 施普林格科学商业媒体出版社, 第 2 版, 2012 年。
- [112] 张 Q., 李 H., 《MOEA/D: 一种基于分解的多目标进化算法》, 《IEEE 进化计算汇刊》第 11 卷第 6 期 (2007 年), 第 712 - 731 页。
- [113] D. 齐默尔曼, 《两个假设同时被违背对参数和非参数统计检验的影响》, 《实验教育杂志》第 67 卷第 1 期 (1998 年), 第 55 - 68 页。
- [114] T. 齐默尔曼、P. 魏斯格伯、S. 迪尔、A. 策勒, 《挖掘版本历史以指导软件变更》, 载于《第 26 届软件工程国际会议论文集》, ICSE ' 04, 美国电气和电子工程师协会计算机学会, 美国华盛顿特区, 国际标准书号 0 - 7695 - 2163 - 0, 第 563 - 572 页, 网址 <http://dl.acm.org/citation.cfm?id=998675.999460>, 2004 年。